Laboratory Exercise 4
Decoders, Code Converters and Seven-Segment Displays

Introduction

This laboratory exercise accompanies the first half of Lecture 5 (Combinational Circuit Building Blocks): the design procedure, decoders, encoders, code converters and the BCD (decimal) adder. All circuits are written in VHDL and tested on the DE0-CV board. You will learn how to use the `WITH SELECT` and `WHEN ... ELSE` statements to describe truth tables directly, how to combine small entities into larger ones, and how to display numbers on the seven-segment displays. The `hex7seg` decoder that you write in Part II will be reused in later labs (counters and microprocessors).

Objectives

1. To describe decoders and encoders in VHDL using selected and conditional signal assignments.
2. To build a larger decoder from smaller decoders with enable inputs.
3. To design a binary-to-seven-segment decoder and use it to display hexadecimal digits.
4. To design a BCD (decimal) adder using binary adders and a correction circuit.
5. To use the standard top-level entity template of the DE0-CV board.

Equipment Required

- Intel DE0-CV board and a PC with Quartus Prime
- Files from Lab 3: `full_adder.vhd` and `ripple_adder_4bit.vhd`

Pre-lab (complete before the session)

The session is two hours long, so complete the following *before* the session.

1. Write the truth table of a 2-to-4 decoder with active-high outputs and an enable input.
2. Using Figure 1, fill in Table 1 for the digits 0 to F (segment pattern with **1 = segment ON**). The DE0-CV displays are **active-low**: a segment is lit when its control signal is 0. Two rows are given as examples.

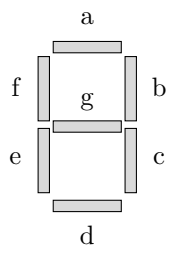


Figure 1: Segments of a seven-segment display. On the DE0-CV board, `HEX0[0]` drives segment a, `HEX0[1]` drives b, ..., `HEX0[6]` drives g.

Digit	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2							
3							
4							
5							
6							
7							
8							
9							
A							
b							
C							
d							
E							
F							

Table 1: Segment patterns (1 = segment ON).

Standard Top-Level Template

From this lab on, the circuit that is downloaded to the board is described by a **top-level entity** whose ports are named like the board pins (`SW`, `KEY`, `LEDR`, `HEX0`, ...). The pin assignments can then be imported from the file supplied by your instructor (or from the DE0-CV User Manual) once, and reused in all later projects. The top-level entity simply connects the board pins to your own entities. Remember that `KEY` is active-low.

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY lab4_top IS
    PORT ( SW   : IN  STD_LOGIC_VECTOR(9 DOWNTO 0);
           KEY  : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
           LEDR : OUT STD_LOGIC_VECTOR(9 DOWNTO 0);
           HEX0 : OUT STD_LOGIC_VECTOR(6 DOWNTO 0);
           HEX1 : OUT STD_LOGIC_VECTOR(6 DOWNTO 0) );
END lab4_top;

ARCHITECTURE structural OF lab4_top IS
BEGIN
    -- instantiate your entities here and connect them to SW, KEY, LEDR, HEX
END structural;
```

Figure 2: Top-level template for the DE0-CV board (add or remove ports as needed).

Part I: Decoders (about 25 min)

A 2-to-4 decoder with enable activates one of four outputs. With selected signal assignment, a truth table can be written directly in VHDL (Figure 3). The operator `&` joins signals into a longer vector (concatenation).

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dec2to4 IS
    PORT ( A  : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
           EN : IN  STD_LOGIC;
           Y  : OUT STD_LOGIC_VECTOR(3 DOWNTO 0) );
END dec2to4;

ARCHITECTURE behavior OF dec2to4 IS
    SIGNAL sel : STD_LOGIC_VECTOR(2 DOWNTO 0);
BEGIN
    sel <= EN & A;
    WITH sel SELECT
        Y <= "0001" WHEN "100",
             "0010" WHEN "101",
             "0100" WHEN "110",
             "1000" WHEN "111",
             "0000" WHEN OTHERS;
END behavior;
```

Figure 3: A 2-to-4 decoder with enable.

1. Create a new Quartus project `lab4`, type the code of Figure 3 and simulate it for all combinations of EN and A. Does the result agree with your pre-lab truth table?

2. **3-to-8 decoder.** Write an entity `dec3to8` with inputs $A(2..0)$, EN and output $Y(7..0)$ using two instances of `dec2to4`. The lower decoder is enabled when $EN = 1$ and $A_2 = 0$; the upper decoder when $EN = 1$ and $A_2 = 1$. Connect the outputs as $Y(3..0)$ and $Y(7..4)$. Draw the block diagram in your report.

3. Simulate the 3-to-8 decoder for all 16 input combinations.

4. Connect it to the board: A = SW2–SW0, EN = SW9 and Y = LEDR7–LEDR0. Verify that only one LED is lit and that no LED is lit when $EN = 0$.

Part II: Binary-to-Seven-Segment Decoder (about 30 min)

The seven-segment decoder is used to display a 4-bit number on a HEX display. Its truth table (the entries of Table 1, but **inverted** because the displays are active-low) is written with a `WITH SELECT` statement. The segments are numbered `seg(0)` $= a$, ..., `seg(6)` $= g$, so the bit pattern is written as "$gfedcba$".

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY hex7seg IS
    PORT ( hex : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
           seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0) );   -- active low, seg(0)=a ... seg(6)=g
END hex7seg;

ARCHITECTURE behavior OF hex7seg IS
BEGIN
    WITH hex SELECT
        --          gfedcba
        seg <= "1000000" WHEN "0000",   -- 0
               "1111001" WHEN "0001",   -- 1
               -- TODO: complete the table for 2 ... F
               "1111111" WHEN OTHERS;   -- all segments off
END behavior;
```

Figure 4: Skeleton of the hexadecimal seven-segment decoder.

1. Complete the `hex7seg` entity using the patterns of Table 1. Remember to **invert** the patterns.
2. Write a top-level entity using the template of Figure 2: connect SW3–SW0 to `hex7seg` and the output to `HEX0`. Compile and download. Check that the display shows 0 to F correctly as you step through the switches.
3. Extend the top-level so that SW7–SW4 are displayed on `HEX1`.

Part III: Decimal (BCD) Adder (about 40 min)

A BCD adder adds two decimal digits A and B (each 0–9 coded in 4 bits) and produces a decimal result in the range 0–18, displayed as two digits. As explained in Lecture 5, the 4-bit binary sum $Z = A + B$ must be corrected by adding 6 (0110_2) whenever it exceeds 9, and the carry then becomes the tens digit:

$$K = C_{out} + Z_3 Z_2 + Z_3 Z_1$$

where C_{out} is the carry-out of the first adder, and the final result is $S = Z + 0KK0$, with K also the tens digit.

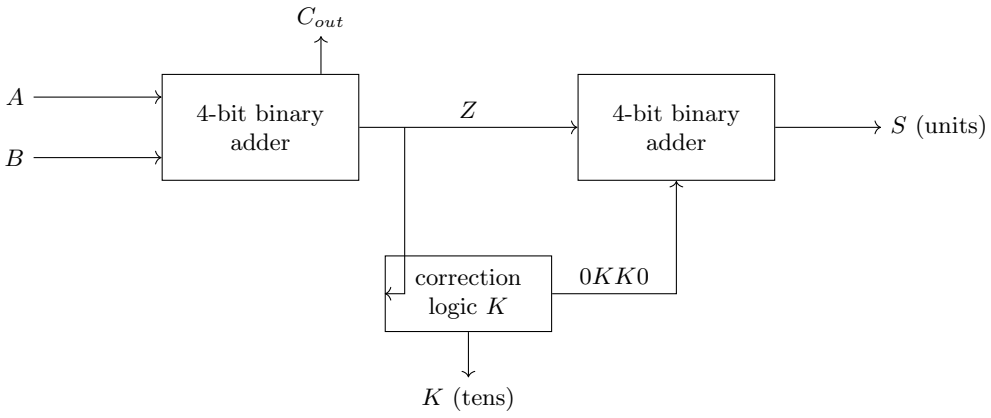


Figure 5: Block diagram of the BCD adder.

1. Write an entity `bcd_adder` that uses two instances of your `ripple_adder_4bit` from Lab 3 and the correction logic of the equation above. Hints: use `Cin => '0'` for both adders and `Cout => OPEN` for the unused carry output; the correction value is formed with the concatenation `"0" & K & K & "0"`.

2. Simulate the adder with $A + B$ equal to $0 + 0$, $4 + 5$, $9 + 1$, $9 + 9$ and $5 + 7$. Verify that the digits are correct.

3. Make a top-level entity: A = SW3–SW0, B = SW7–SW4, the units digit on `HEX0` and the tens digit on `HEX1` (using `hex7seg`).

4. Test the circuit on the board and record the results in Table 2 (the last row checks what happens with an invalid BCD digit).

A	B	Tens (HEX1)	Units (HEX0)	Comment
0	0			
4	5			
9	1			
9	9			
7	8			
12	3			

Table 2: BCD adder results on the board.

Extension (take-home, or if time permits)

1. **Function generator (Part I).** A decoder produces all the minterms of its inputs, so any function can be built with an OR gate behind it. Using the 3-to-8 decoder, implement the sum S and carry C_{out} of a full adder (A, B, C_{in} = SW2–SW0):

   $$S = \sum m(\ _____\ ) \qquad C_{out} = \sum m(\ _____\ )$$

   Write the two OR expressions, for example `S <= Y(1) OR Y(2) OR ...;`, and show S, C_{out} on LEDR9 and LEDR8.

2. (Part II) Modify `hex7seg` so that in a BCD application (digits 0–9) the codes 1010 to 1111 display a blank. Is it still necessary to specify these codes if they are "don't cares"? Compare the number of logic elements used for the two versions. _____

3. (Part III) Add an **invalid-input indicator**: LEDR0 must be lit when $A > 9$ or $B > 9$. Write the Boolean expression for $A > 9$ using the bits A_3, A_2, A_1: _____

4. **Priority encoder.** A priority encoder produces the binary index of the highest-priority active input (I_3 has the highest priority) and a "valid" output V that is 1 when at least one input is active. Write the entity `prio_enc4` with input $I(3..0)$ and outputs $Y(1..0)$ and V with a conditional assignment (`Y <= "11" WHEN I(3) = '1' ELSE "10" WHEN I(2) = '1' ELSE ...;` and `V <= I(3) OR I(2) OR I(1) OR I(0);`). Test it on the board with I = SW3–SW0, Y on `HEX0` and V on LEDR0. What is displayed for $I = 0101$ and for $I = 0000$? Why is V needed?

Check-off

Show your instructor or teaching assistant:

1. The 3-to-8 decoder (and the full adder built from it) working on the board.
2. The hexadecimal display 0–F on `HEX0` and `HEX1`.
3. The BCD adder with the invalid-input indicator (Table 2).

Updated By: R. Sutthaweekul
Version: 1.0
Release Date: 2026-11-11